



Custom Examples in daltoolbox

A practical guide to extending **daltoolbox** with external backends. The examples cover the three extension roles — preprocessing, supervised learning, and clustering — showing how the integration contract remains small and uniform across all cases.

R · DALTOOLBOX

CUSTOM EXTENSIONS

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br

<https://eic.cefet-rj.br/~eogasawara>

Custom Examples in daltoolbox

The **daltoolbox** keeps the workflow stable while the user connects a new backend through a small integration contract. This series of examples organizes extensions by role: preprocessing, supervised learning, and unsupervised components.

01

Data Preparation

Custom transformations before modeling

02

Supervised Learning

Custom classifiers and regressors

03

Unsupervised Learning

Clustering with external backends

Custom Transformation: `smote_custom`

SMOTE (Synthetic Minority Over-sampling Technique) is a balancing algorithm that generates synthetic samples for minority classes through **interpolation** between existing minority instances and their k -nearest neighbors in the feature space — rather than simply duplicating samples. This avoids overfitting and produces more realistic decision boundaries.

① Select Sample

A minority class instance is randomly chosen as the starting point.

② Find k -Neighbors

The k nearest neighbors of that instance are identified in the feature space.

③ Interpolate Synthetic Point

A new point is generated along the line segment between the sample and a randomly selected neighbor: $x_{\text{new}} = x + \lambda \cdot (x_{\text{nn}} - x)$, with $\lambda \in [0,1]$.

Integration Contract

- Constructor: `smote_custom(attribute)`
- Method: `transform.smote_custom(obj, data, ...)`
- External backend: `smotefamily::SMOTE`

Result on Imbalanced Iris

Before: setosa=50, versicolor=21, virginica=11

After: setosa=50, versicolor=42, virginica=44

- ① Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.

Custom Data Preparation: smote_custom

```
library(daltoolbox)

smote_custom <- function(attribute) {
  obj <- daltoolbox::dal_transform()
  obj$attribute <- attribute
  class(obj) <- append("smote_custom",
                       class(obj))

  obj
}

transform.smote_custom <- function(
  obj, data, ...) {
  data <- data.frame(data,
                     check.names = FALSE)
  j <- match(obj$attribute,
             colnames(data))
  x <- sort(table(
    data[, obj$attribute]))
  # oversampling loop per class
  # using smotefamily::SMOTE
  ...
}
```

```
iris_imb <- datasets::iris[
  c(1:50, 51:71, 101:111), ]
bal <- smote_custom("Species")
iris_bal <- transform(bal, iris_imb)
table(iris_bal$Species)
# setosa:50 versicolor:42 virginica:44
```

Step by Step

- **Constructor:** calls `dal_transform()`, stores the target attribute and adds the custom class.
- **S3 Method:** implements `transform.smote_custom()` — the only method required by the contract.
- **External backend:** delegates oversampling to `smotefamily::SMOTE`, outside the `daltoolbox` namespace.
- **Result:** balanced classes (50 / 42 / 44) without altering the rest of the pipeline.

📌 **Key concept:** The integration contract is minimal — just a constructor + S3 method. The analytical workflow remains unchanged.

Custom Classification: `cla_rsnns_custom`

A **Multilayer Perceptron (MLP)** is a feedforward artificial neural network composed of an input layer, one or more hidden layers, and an output layer. Each neuron applies a **non-linear activation function** (e.g., sigmoid or ReLU) to a weighted sum of its inputs. Training uses **backpropagation** with gradient descent to minimize a loss function. MLPs are universal function approximators (Cybenko, 1989) and are widely used in classification tasks.



Input Layer

Receives the feature vector (e.g., 4 attributes from Iris) and propagates them to the next layer.



Output Layer

Produces per-class probabilities via **softmax**. The class with the highest probability is the final prediction.

Hyperparameters

- `size` — neurons in the hidden layer (default: 5)
- `learn_rate` — learning rate (default: 0.2)
- `maxit` — maximum iterations (default: 100)



Hidden Layer(s)

Applies synaptic weights + sigmoid activation function: $\sigma(z) = \frac{1}{1+e^{-z}}$. The `size` hyperparameter defines the number of neurons.



Backpropagation

Adjusts weights by minimizing the **cross-entropy loss** via gradient descent. Controlled by `learn_rate` and `maxit`.

Training (Iris)

Accuracy: **0.9583**
Precision, Recall, F1: **1.0**

Test (Iris)

Accuracy: **0.9333**
Precision, Recall, F1: **1.0**

- ❏ **References:** Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. — Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.

Custom Classifier: `cla_rsnns_custom`

```
library(daltoolbox)

cla_rsnns_custom <- function(
  attribute, slevels,
  size = 5, learn_rate = 0.2,
  maxit = 100) {
  obj <- daltoolbox::classification(
    attribute, slevels)
  obj$size <- size
  obj$learn_rate <- learn_rate
  obj$maxit <- maxit
  class(obj) <- append(
    "cla_rsnns_custom", class(obj))
  obj
}

fit.cla_rsnns_custom <- function(
  obj, data, ...) {
  x <- as.matrix(data[, obj$x,
    drop = FALSE])
  y <- daltoolbox::adjust_class_label(
    data[, obj$attribute])
  obj$model <- RSNNS::mlp(
    x = x, y = y,
    size = obj$size,
    learnFuncParams = c(obj$learn_rate),
    maxit = obj$maxit)
  obj
}
```

```
predict.cla_rsnns_custom <- function(
  object, x, ...) {
  x <- as.matrix(x[, object$x,
    drop = FALSE])
  prediction <- predict(
    object$model, x)
  colnames(prediction) <- object$selevels
  as.data.frame(prediction)
}

# Train / Test
model <- cla_rsnns_custom(
  "Species", slevels,
  size = 5, learn_rate = 0.1,
  maxit = 150)
model <- fit(model, iris_train)
# accuracy train: 0.9583
# accuracy test: 0.9333
```

Step by Step

- **Constructor:** inherits from `classification()` and stores hyperparameters (`size`, `learn_rate`, `maxit`).
- **fit():** prepares X and Y matrices, trains the MLP via `RSNNS::mlp` and returns the updated object.
- **predict():** returns a `data.frame` with columns named by levels — the format expected by `evaluate()`.
- **Downstream:** sampling and evaluation are identical to native `daltoolbox` examples.

❏ **Key concept:** Constructor + fit() + predict() is the complete contract. The algorithm complexity stays in the backend; the integration remains small and predictable.

Custom Regression: `reg_rsnns_custom`

To adapt an MLP for regression, the output layer uses **linear activation** (`linOut = TRUE` in RSNNS), allowing the network to predict continuous values without restricting the output to the `[0,1]` interval. The loss function changes from **cross-entropy** (classification) to **Mean Squared Error (MSE)**. The hidden layer architecture remains identical to the classification MLP — only the output contract and loss function differ.

Output Activation

Classification: softmax

Regression: linear (identity)

Return Type

Classification: probabilities per class

Regression: continuous numeric vector

Loss Function

Classification: cross-entropy

Regression: MSE (mean squared error)

RSNNS Parameter

Classification: `linOut = FALSE`

Regression: `linOut = TRUE`

Integration Contract

- **Constructor:** `reg_rsnns_custom(attribute, size, learn_rate, maxit)`
- **fit():** trains MLP with `linOut = TRUE` via `RSNNS::mlp`
- **predict():** returns a numeric vector (no class columns)

Results – Boston Dataset

Train: MSE = 108.64 · SMAPE = 0.369 · R^2 = -0.207

Test: MSE = 95.48 · SMAPE = 0.378 · R^2 = -0.587

The negative R^2 values indicate that the model, without hyperparameter tuning, performs below a simple baseline — expected for a first standard run.

- 📖 **References:** Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. — Bergmeir, C., and Benitez, J. M. (2012). Neural Networks in R Using the Stuttgart Neural Network Simulator: RSNNS. *Journal of Statistical Software*, 46(7), 1–26.

Custom Regressor: reg_rsnns_custom

```
library(daltoolbox)
reg_rsnns_custom <- function(
  attribute, size = 5,
  learn_rate = 0.1, maxit = 200) {
  obj <- daltoolbox::regression(
    attribute)
  obj$size <- size
  obj$learn_rate <- learn_rate
  obj$maxit <- maxit
  class(obj) <- append(
    "reg_rsnns_custom", class(obj))
  obj
}
fit.reg_rsnns_custom <- function(
  obj, data, ...) {
  x <- as.matrix(data[, obj$x,
    drop = FALSE])
  y <- as.matrix(data[, obj$attribute,
    drop = FALSE])
  obj$model <- RSNNS::mlp(
    x = x, y = y,
    size = obj$size,
    learnFuncParams = c(obj$learn_rate),
    maxit = obj$maxit,
    linOut = TRUE) # linear output!
  obj
}
```

```
predict.reg_rsnns_custom <- function(
  object, x, ...) {
  x <- as.matrix(x[, object$x,
    drop = FALSE])
  as.numeric(predict(object$model, x))
}

# Boston Dataset (MASS)
model <- reg_rsnns_custom(
  "medv", size = 6,
  learn_rate = 0.05, maxit = 200)
model <- fit(model, boston_train)
# MSE train: 108.64
# MSE test: 95.48
```

Step by Step

- **Constructor:** inherits from `regression()`; hyperparameters are stored in the object.
- **fit():** uses `linOut = TRUE` to enable continuous linear output — the essential difference from the classifier.
- **predict():** returns a simple numeric vector, compatible with `evaluate()`.
- **Separation of concerns:** the algorithm can be complex; the integration contract remains identical to that of the classifier.

❏ **Key concept:** The only structural difference between a custom classifier and a custom regressor is `linOut = TRUE` and the return type of `predict()`.

Custom Clustering: `cluster_agnes_custom`

AGNES (Agglomerative Nesting) is a *bottom-up* hierarchical clustering algorithm. It starts with each observation as its own cluster and iteratively merges the two most similar clusters until only a single cluster remains. The result is a **dendrogram** — a tree structure that can be cut at any level to produce k clusters. The linkage method (e.g., Ward) determines how the distance between clusters is computed.



1 • Initialize

Each observation forms its own cluster — n clusters for n points.



2 • Compute Distances

Builds the distance matrix using the chosen metric (e.g., Euclidean).



3 • Merge

Joins the two closest clusters according to the linkage criterion — Ward minimizes intra-cluster variance.



4 • Repeat

Recomputes distances and repeats the merge until all observations belong to a single cluster.



5 • Cut

Applies `cutree(k)` to extract k flat clusters from the dendrogram.

Integration Contract

- **Constructor:** inherits from `clustering()`; stores k and the linkage method.
- **`fit()`:** calls `cluster::agnes()` and stores the learned hierarchical structure.
- **`cluster()`:** applies `cutree()` and returns a vector of integer labels, compatible with `evaluate()`.

Results — Iris ($k=3$, Ward)

- Cluster 1: **49** observations
- Cluster 2: **30** observations
- Cluster 3: **71** observations

Entropy per cluster: $CE_1=0.000$ · $CE_2=0.561$ · $CE_3=0.909$

Total entropy: **0.542**

❏ **References:** Kaufman, L., and Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley. — Ward, J. H. (1963). Hierarchical Grouping to Optimize an Objective Function. *Journal of the American Statistical Association*, 58(301), 236–244.

Custom Clusterer: `cluster_agnes_custom`

```
library(daltoolbox)

cluster_agnes_custom <- function(
  k = 3, method = "ward",
  metric = "euclidean",
  scale = TRUE) {
  obj <- daltoolbox::clusterer()
  obj$k <- k
  obj$method <- method
  obj$metric <- metric
  obj$scale <- scale
  class(obj) <- append(
    "cluster_agnes_custom", class(obj))
  obj
}

fit.cluster_agnes_custom <- function(
  obj, data, ...) {
  x <- as.matrix(data)
  storage.mode(x) <- "double"
  if (isTRUE(obj$scale)) x <- scale(x)
  obj$model <- cluster::agnes(
    x, diss = FALSE,
    metric = obj$metric,
    method = obj$method)
  obj
}
```

```
cluster.cluster_agnes_custom <- function(
  obj, data, ...) {
  x <- as.matrix(data)
  storage.mode(x) <- "double"
  clu <- stats::cutree(
    as.hclust(obj$model), k = obj$k)
  # calcula métrica intra-cluster
  ...
  attr(clu, "metric") <- dist
  clu
}

# Iris — 4 features, k=3
model <- cluster_agnes_custom(
  k = 3, method = "ward")
model <- fit(model, iris[, 1:4])
clu <- cluster(model, iris[, 1:4])
# clu: 1→49 2→30 3→71
eval <- evaluate(model, clu,
  iris$Species)
# clustering_entropy: 0.542
```

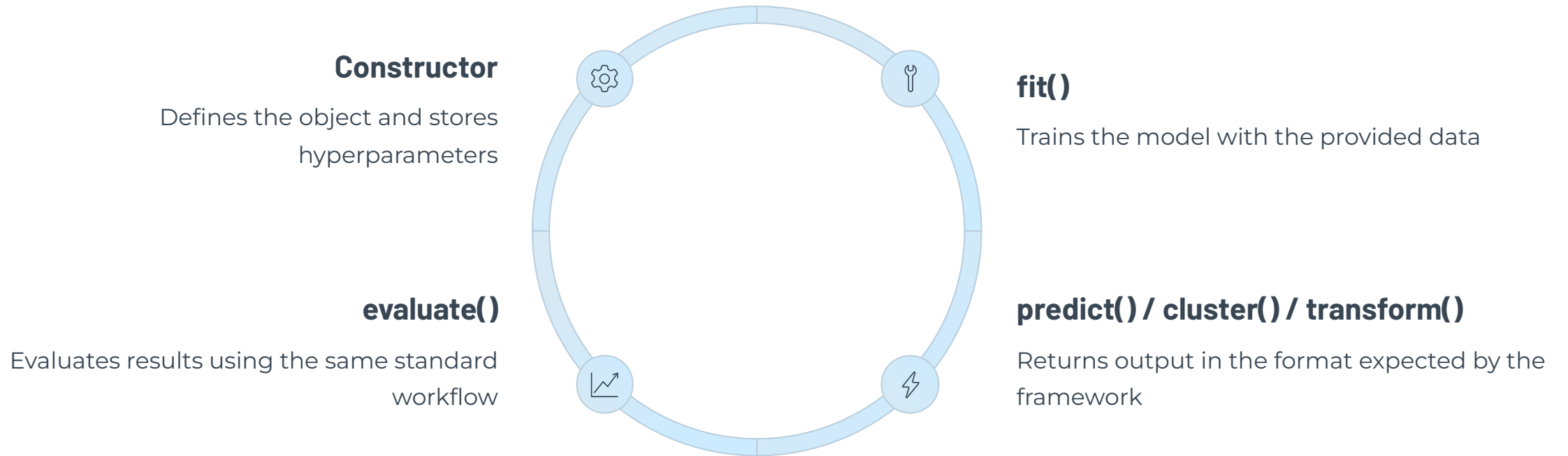
Step by Step

- **Constructor:** inherits from `clusterer()`; stores `k`, linkage method, and distance metric.
- **fit():** scales the data (optional) and trains the agglomerative hierarchical model via `cluster::agnes`.
- **cluster():** cuts the dendrogram with `cutree()` and computes the intra-cluster distance as a quality metric.
- **evaluate():** returns per-cluster entropy and global entropy — same standard as native clusterers.

❏ **Key concept:** The contract for clustering is constructor + fit() + cluster(). The workflow remains stable even with an external algorithm.

The Framework's Unifying Principle

Regardless of the extension type — transformation, classification, regression, or clustering — daltoolbox maintains the same small and uniform integration contract.



❏ **Conclusion:** The mining method can be sophisticated, but the integration contract remains small, stable, and reusable at any point in the Experiment Line.

Summary: The daltoolbox Integration Contract

Across all examples — **SMOTE** (data preparation), **MLP via RSNNS** (classification and regression), and **AGNES** (clustering) — the unifying idea is the same: the mining method can be sophisticated, but the integration with the framework remains small and uniform.

Transformation

- Constructor
- `transform()`

`smote_custom` — generates synthetic minority samples via SMOTE interpolation

Classification / Regression

- Constructor
- `fit()`
- `predict()`

`cla_rsnns_custom` / `reg_rsnns_custom` — MLP via RSNNS; only `linOut=TRUE` distinguishes regressor from classifier

Clustering

- Constructor
- `fit()`
- `cluster()`

`cluster_agnes_custom` — AGNES hierarchical clustering with `cutree(k)`

□ **Core principle:** Three external backends — `smotefamily` (SMOTE), `RSNNS` (MLP), and `cluster::agnes` (hierarchical clustering) — each encapsulate algorithmic complexity while integrating cleanly into the Experiment Line. Sampling, training, and evaluation remain identical regardless of which extension is used.

References & Additional Sources

Academic references and online resources to explore the DAL Toolbox and Experiment Lines further.

Online Resources

CRAN Package – daltoolbox

cran.r-project.org/web/packages/daltoolbox

GitHub Repository

github.com/cefet-rj-dal/daltoolbox

Source code, issues, and contributions.

Key Academic References

1. Ogasawara, E., Sa, A., Castro, A., Santos, C., Salles, D., Lima, J., Tavares, L., Salles, R. & Saidy, V. (2026). daltoolbox: Leveraging Experiment Lines to Data Analytics. R package version 1.3.767. *CRAN*.
2. Ogasawara, E., Paulino, C., Murta, L., Werner, C. & Mattoso, M. (2009). Experiment Line: Software Reuse in Scientific Workflows. *SSDBM 2009, LNCS 5566*, pp. 264–272. *Springer*.
3. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
4. He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
5. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
6. Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.
7. Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366.
8. Bergmeir, C., & Benítez, J. M. (2012). Neural Networks in R Using the Stuttgart Neural Network Simulator: RSNNS. *Journal of Statistical Software*, 46(7), 1–26.
9. Kaufman, L., & Rousseeuw, P. J. (1990). Finding Groups in Data: An Introduction to Cluster Analysis. *Wiley*.
10. Ward, J. H. (1963). Hierarchical Grouping to Optimize an Objective Function. *Journal of the American Statistical Association*, 58(301), 236–244.